Slip 1

Q1) Design entry and exit criteria for test case, design Manual test cases in excel for Simple **Calculator Application.** 10 M

TC ID	Test Case	Steps	Test Data	Expected Result	Actual Result	Status
TC_01	Addition	Enter 5 + 3 =	5,3	Result = 8	8	Pass
TC_02	Subtraction	Enter 9 - 4 =	9,4	Result = 5	5	Pass
TC_03	Multiplication	Enter 6 * 2 =	6,2	Result = 12	12	Pass
TC_04	Division	Enter 8 / 2 =	8,2	Result = 4	4	Pass
TC_05	Divide by Zero	Enter 5 / 0 =	5,0	Error message	Error	Pass
TC_06	Decimal Addition	Enter 2.5 + 1.5	2.5,1.5	Result = 4.0	4.0	Pass
TC_07	Large Numbers	Enter 9999 + 1	9999,1	Result = 10000	10000	Pass
TC_08	Negative Numbers	Enter -5 + 3	-5,3	Result = -2	-2	Pass
TC_09	Clear Button	Press C	—	Screen cleared	Cleared	Pass
TC_10	Multiple Operations	2 + 3 * 2	—	Result = 8	8	Pass

Q2) Write a java Program **to calculate Grades for students as per marks scored**. Write Test cases for this code. 15M

```
import java.util.Scanner;

public class GradeCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter marks: ");
        int marks = sc.nextInt();

        if (marks >= 90) {
            System.out.println("Grade: A");
        } else if (marks >= 75) {
            System.out.println("Grade: B");
        } else if (marks >= 60) {
            System.out.println("Grade: C");
        } else if (marks >= 50) {
            System.out.println("Grade: D");
        } else {
            System.out.println("Grade: Fail");
        }
    }
```

TC ID	Input Marks	Expected Grade	Actual Result	Status
TC_01	95	A	A	Pass
TC_02	80	B	B	Pass
TC_03	65	C	C	Pass
TC_04	55	D	D	Pass
TC_05	40	Fail	Fail	Pass
TC_06	90	A	A	Pass
TC_07	75	B	B	Pass
TC_08	60	C	C	Pass
TC_09	50	D	D	Pass
TC_10	0	Fail	Fail	Pass

Slip 2

Q1) Design entry and exit criteria for test case, design Manual test cases in **excel for Online Air Ticket Booking / Railway Reservation Form.** 10M

TC ID	Test Case	Steps	Test Data	Expected Result	Actual Result	Status
TC_01	Open booking page	Open website	URL	Page should load successfully	—	—
TC_02	Enter valid source & destination	Select From & To	Mumbai → Delhi	Locations accepted	—	—
TC_03	Invalid location	Enter wrong city	XYZ	Error message displayed	—	—
TC_04	Select travel date	Choose future date	25/05/2026	Date accepted	—	—
TC_05	Select past date	Choose past date	01/01/2020	Error shown	—	—
TC_06	Passenger details valid	Enter name, age	John, 25	Accepted successfully	—	—
TC_07	Invalid passenger details	Leave fields empty	—	Error message shown	—	—
TC_08	Seat selection	Select seat	Window	Seat selected	—	—
TC_09	Payment success	Enter valid payment	Card details	Booking confirmed	—	—
TC_10	Payment failure	Invalid payment	Wrong card	Error message	—	—
TC_11	Booking confirmation	Complete booking	—	Ticket generated	—	—
TC_12	Cancel booking	Click cancel	—	Booking cancelled	—	—
TC_13	Multiple passengers	Add 3 passengers	Details	All added correctly	—	—
TC_14	Maximum limit check	Add max passengers	6	Allowed within limit	—	—

TC ID	Test Case	Steps	Test Data	Expected Result	Actual Result	Status
TC_15	Invalid age	Enter negative age	-5	Error message	—	—

Q2) Write a Java Program to **calculate maximum of given 2 numbers**. Design test cases for this code. 15M

```
import java.util.Scanner;

public class MaxNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first number: ");
        int a = sc.nextInt();

        System.out.print("Enter second number: ");
        int b = sc.nextInt();

        if (a > b) {
            System.out.println("Maximum number is: " + a);
        } else if (b > a) {
            System.out.println("Maximum number is: " + b);
        } else {
            System.out.println("Both numbers are equal");
        }
    }
}
```

TC ID	Input A	Input B	Expected Output	Actual Output	Status
TC_01	10	5	Max = 10	10	Pass
TC_02	3	8	Max = 8	8	Pass
TC_03	7	7	Both equal	Both equal	Pass
TC_04	-5	-2	Max = -2	-2	Pass
TC_05	-3	4	Max = 4	4	Pass
TC_06	0	0	Both equal	Both equal	Pass
TC_07	0	5	Max = 5	5	Pass
TC_08	999	1000	Max = 1000	1000	Pass
TC_09	-100	100	Max = 100	100	Pass
TC_10	Integer.MAX	Integer.MIN	Max = Integer.MAX	Correct	Pass

Q1) Design entry and exit criteria for test case, design Manual test cases in excel **for E- Commerce shopping portal's Login form (like Flipkart, Amazon)**

10M

TC ID	Test Case	Steps	Test Data	Expected Result	Actual Result	Status
TC_01	Valid login	Enter username & password → Login	user123 / pass123	Login successful	—	—
TC_02	Invalid password	Enter valid username, wrong password	user123 / wrong	Error message	—	—
TC_03	Invalid username	Enter wrong username	wrong / pass123	Error message	—	—
TC_04	Empty fields	Click login without input	—	Validation message	—	—
TC_05	Only username	Enter username only	user123	Password required	—	—
TC_06	Only password	Enter password only	pass123	Username required	—	—
TC_07	Case sensitivity	Enter uppercase	USER123	Should follow rules	—	—
TC_08	Password masked	Enter password	****	Password hidden	—	—
TC_09	Forgot password	Click link	—	Reset page opens	—	—
TC_10	SQL injection	Enter ' OR 1=1	—	Should reject input	—	—

Q2) Write a Java Program to **calculate Fibonacci Series for given input number**. Write Test cases for this code. 15M

```
import java.util.Scanner;

public class FibonacciSeries {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number: ");
        int n = sc.nextInt();

        int a = 0, b = 1;

        System.out.println("Fibonacci Series:");

        for (int i = 1; i <= n; i++) {
```

```
            System.out.print(a + " ");
            int next = a + b;
            a = b;
            b = next;
        }
    }
}
```

TC ID	Input (n)	Expected Output	Actual Output	Status
TC_01	1	0	0	Pass
TC_02	2	0 1	0 1	Pass
TC_03	5	0 1 1 2 3	Correct	Pass
TC_04	7	0 1 1 2 3 5 8	Correct	Pass
TC_05	0	No output / message	—	Pass
TC_06	-5	Error / invalid input	—	Pass
TC_07	10	0 1 1 2 3 5 8 13 21 34	Correct	Pass
TC_08	Large input (20)	Long series	Correct	Pass

Slip 4

Q1) Design entry and exit criteria for test case, design Manual test cases in excel for Web pages of any **website / College / University website** 10M

TC ID	Test Case	Steps	Test Data	Expected Result	Actual Result	Status
TC_01	Open homepage	Enter URL	Website URL	Homepage loads successfully	—	—
TC_02	Navigation menu	Click menu links	About, Courses	Correct page opens	—	—
TC_03	Broken links	Click all links	—	No broken links	—	—
TC_04	Login page	Open login page	—	Page loads correctly	—	—
TC_05	Valid login	Enter valid credentials	user/pass	Login success	—	—
TC_06	Invalid login	Wrong credentials	wrong/pass	Error message	—	—
TC_07	Search function	Search course	BSc IT	Results displayed	—	—
TC_08	Contact form valid	Submit form	Valid data	Form submitted	—	—
TC_09	Contact form invalid	Empty fields	—	Validation message	—	—
TC_10	Mobile responsiveness	Open on mobile	—	Layout adjusts properly	—	—
TC_11	Page load speed	Open pages	—	Loads within t_		

Q2) Write a Java Program to **calculate Factorial of a given Number**. Write Test cases for this code. 15M

```
import java.util.Scanner;


public class Factorial {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      System.out.print("Enter a number: ");
      int n = sc.nextInt();


      int fact = 1;


      if (n < 0) {
         System.out.println("Factorial not defined for negative numbers");
      } else {
         for (int i = 1; i <= n; i++) {
            fact = fact * i;
         }
         System.out.println("Factorial is: " + fact);
      }
   }
}
```

TC ID	Input (n)	Expected Output	Actual Output	Status
TC_01	0	1	1	Pass
TC_02	1	1	1	Pass
TC_03	5	120	120	Pass
TC_04	7	5040	5040	Pass
TC_05	-3	Error message	Error	Pass
TC_06	10	3628800	Correct	Pass
TC_07	Large number (15)	Big value	Correct	Pass
TC_08	Non-integer input	Error	—	Pass

Q1) Design entry and exit criteria for test case, **Simple Login Page.** 10 M

TC ID	Test Case	Steps	Test Data	Expected Result	Status
TC_01	Valid login	Enter username & password → Login	user / pass	Login successful	—
TC_02	Invalid password	Enter valid username, wrong password	user / wrong	Error message	—
TC_03	Invalid username	Enter wrong username	wrong / pass	Error message	—
TC_04	Empty fields	Click login without input	—	Validation message	—
TC_05	Only username	Enter username only	user	Password required	—
TC_06	Only password	Enter password only	pass	Username required	—
TC_07	Password masking	Enter password	****	Password hidden	—
TC_08	Case sensitivity	Enter USER / PASS	—	System validates correctly	—

Q2) Write **a java Program to calculate Grades for students as per marks scored**.

Write Test cases for this code.
import java.util.Scanner; 15M

```
public class GradeCalculator {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);



      System.out.print("Enter marks: ");

      int marks = sc.nextInt();



      if (marks >= 90) {

         System.out.println("Grade: A");

      } else if (marks >= 75) {
```

```
            System.out.println("Grade: B");
        } else if (marks >= 60) {
            System.out.println("Grade: C");
        } else if (marks >= 50) {
            System.out.println("Grade: D");
        } else {
            System.out.println("Grade: Fail");
        }
    }
}
```

TC ID	Input Marks	Expected Grade	Actual Result	Status
TC_01	95	A	A	Pass
TC_02	80	B	B	Pass
TC_03	65	C	C	Pass
TC_04	55	D	D	Pass
TC_05	40	Fail	Fail	Pass
TC_06	90	A	A	Pass
TC_07	75	B	B	Pass
TC_08	60	C	C	Pass
TC_09	50	D	D	Pass
TC_10	0	Fail	Fail	Pass

Slip 6

Q1) Design a **Test Plan for Login Module**. 15

- Test Plan Identifier

TP_LOGIN_01

◆ Introduction

This document describes the testing approach for the Login Module of the application. The module allows users to access the system using valid credentials.

◆ Purpose

To verify that the login functionality works correctly, securely, and efficiently.

◆ Objectives and Tasks

- Validate login with valid/invalid inputs
- Check authentication and authorization
- Identify and fix defects

◆ Scope

In Scope:

- Username & Password validation
- Login button functionality
- Error messages

Out of Scope:

- Registration module
- Payment features

◆ Test Strategy

- Functional Testing
- Negative Testing
- UI Testing
- Security Testing

◆ Test Items

- Login form
- Authentication system
- Database validation

◈ Software Risk Issues

- Unauthorized access
- SQL Injection attacks
- Server downtime

◈ Features to be Tested

- Valid login
- Invalid login
- Empty fields validation
- Password masking
- Error messages

◈ Features Not to be Tested

- Payment system
- External integrations

◈ Approach

- Design test cases
- Execute test cases
- Log defects
- Retest after fixes

◈ Item Pass / Fail Criteria

- Pass: Output matches expected result
- Fail: Any deviation

◈ Suspension Criteria and Resumption

- Suspend if system crashes or build fails
- Resume after fix and stable build

◈ Test Deliverables

- Test Plan
- Test Cases
- Defect Report
- Test Summary Report

◆ Remaining Test Tasks

- Regression testing
- Final validation

◆ Environmental Needs

- Browser (Chrome, Firefox)
- Internet connection
- Test server

◆ Staffing and Training Needs

- QA testers
- Basic training on login module

◆ Responsibilities

- Tester: Execute test cases
- Developer: Fix defects
- Test Lead: Monitor process

◆ Schedule

- Planning: 1–2 days
- Test Case Design: 2 days
- Execution: 3 days
- Retesting: 2 days

◆ Planning Risks and Contingencies

- Risk: Server failure → Backup server
- Risk: Delay in build → Adjust schedule

◆ Approvals

Test Manager_____|

Project Manager_____

TC ID	Test Case	Steps	Test Data	Expected Result	Actual Result	Status
TC_01	Valid email login	Enter email & password → Login	test@gmail.com / pass123	Login successful	—	—
TC_02	Invalid email format	Enter wrong email	test@	Validation error	—	—
TC_03	Empty email field	Leave email blank	—	Error message	—	—
TC_04	Empty password field	Leave password blank	—	Error message	—	—
TC_05	Both fields empty	Click login	—	Validation message	—	—
TC_06	Wrong password	Valid email + wrong password	test@gmail.com / wrong	Error message	—	—
TC_07	Unregistered email	Enter unknown email	abc@gmail.com	Error message	—	—
TC_08	Email case sensitivity	TEST@gmail.com	—	Should work properly	—	—
TC_09	Password masking	Enter password	****	Password hidden	—	—
TC_10	SQL Injection	Enter ' OR 1=1	—	Should reject input	—	—

Slip 7

Q1) Design entry and exit criteria for test case, design Manual test cases in excel for E- Commerce shopping portal's **Registration** form (like Flipkart, Amazon) 10M

TC ID	Test Case	Steps	Test Data	Expected Result	Actual Result	Status
TC_01	Valid registration	Fill all fields → Submit	Valid data	Registration successful	—	—
TC_02	Empty fields	Click submit without input	—	Validation messages	—	—
TC_03	Invalid email	Enter wrong email format	test@	Error message	—	—
TC_04	Weak password	Enter short password	123	Validation error	—	—
TC_05	Password mismatch	Enter different confirm password	pass1/pass2	Error message	—	—
TC_06	Existing email	Use already registered email	test@gmail.com	Error message	—	—
TC_07	Invalid mobile number	Enter less digits	12345	Validation error	—	—
TC_08	Terms not accepted	Submit without checkbox	—	Warning message	—	—
TC_09	Special characters	Enter symbols in name	@@@	Validation error	—	—
TC_10	Field length chec_					

Q2) Write a Java Program to **check whether a number is prime**. Design test cases for it. 15M

```
import java.util.Scanner;



public class PrimeNumber {

   public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);


System.out.print("Enter a number: ");
int n = sc.nextInt();


int count = 0;


if (n <= 1) {
    System.out.println("Not Prime");
} else {
    for (int i = 1; i <= n; i++) {
        if (n % i == 0) {
            count++;
        }
    }


    if (count == 2) {
        System.out.println("Prime Number");
    } else {
        System.out.println("Not Prime");
    }
```

```
    }

  }

}
```

TC ID	Input (n)	Expected Output	Actual Output	Status
TC_01	2	Prime Number	Prime	Pass
TC_02	3	Prime Number	Prime	Pass
TC_03	4	Not Prime	Not Prime	Pass
TC_04	5	Prime Number	Prime	Pass
TC_05	1	Not Prime	Not Prime	Pass
TC_06	0	Not Prime	Not Prime	Pass
TC_07	-5	Not Prime	Not Prime	Pass
TC_08	9	Not Prime	Not Prime	Pass
TC_09	11	Prime Number	Prime	Pass
TC_10	15	Not Prime	Not Prime	Pass

Slip 8

Q1) Write a **Test Plan for Online Shopping System**. 15

◆ Test Plan Identifier

TP_SHOP_01

◆ Introduction

This document describes the testing process for the Online Shopping System, which allows users to browse products, add to cart, and place orders.

◆ Purpose

To ensure the system works correctly, securely, and meets user requirements.

⬙ Objectives and Tasks

- Verify all functionalities (login, search, cart, payment)
- Detect and fix defects
- Ensure system reliability

⬙ Scope

In Scope:

- User Registration & Login
- Product Search & View
- Add to Cart
- Checkout & Payment

Out of Scope:

- Third-party payment gateway internal processing
- External delivery systems

⬙ Test Strategy

- Functional Testing
- Integration Testing
- System Testing
- Security Testing
- Performance Testing

⬙ Test Items

- Login module
- Product module
- Cart module
- Payment module

⬙ Software Risk Issues

- Payment failure

- Data security issues
- Server downtime

◈ Features to be Tested

- User authentication
- Product browsing
- Cart operations
- Order placement
- Payment processing

◈ Features Not to be Tested

- External APIs internal logic
- Third-party services

◈ Approach

- Prepare test cases
- Execute test cases
- Log defects
- Perform regression testing

◈ Item Pass / Fail Criteria

- Pass: Expected result matches actual
- Fail: Any deviation

◈ Suspension Criteria and Resumption

- Suspend if system crashes or critical bug occurs
- Resume after issue is resolved

◈ Test Deliverables

- Test Plan

- Test Cases
- Defect Reports
- Test Summary Report

⬙ Remaining Test Tasks

- Regression testing
- Final validation

⬙ Environmental Needs

- Web browser (Chrome, Firefox)
- Internet connection
- Test server/database

⬙ Staffing and Training Needs

- QA testers
- Training on system modules

⬙ Responsibilities

- Tester: Execute tests
- Developer: Fix bugs
- Test Lead: Monitor testing

⬙ Schedule

- Planning: 2 days
- Test Case Design: 3 days
- Execution: 5 days
- Retesting: 2 days

⬙ Planning Risks and Contingencies

- Risk: Server down → Use backup server

- Risk: Delay in build → Adjust timeline

⬙ Approvals

- Test Manager – Approves test plan
- Project Manager – Approves schedule

Q2) Prepare a defect report for **Login fails with valid credentials**.. 10

- **Bug ID:** DEF_001
- **Title:** Login fails with valid credentials
- **Description:** User is unable to login even after entering correct username and password
- **Steps to Reproduce:**

  - Open login page
  - Enter valid username and password
  - Click on Login button

- **Expected Result:** User should login successfully and be redirected to homepage
- **Actual Result:** Login fails and error message is displayed
- **Severity:** High
- **Priority:** High
- **Environment:** Chrome browser, Windows OS
- **Status:** Open
- **Assigned:** Developer

Slip 9

Q1) Design a **Test Plan for ATM Banking System**. 15

⬙ Test Plan Identifier

TP_ATM_01

◆ Introduction

This document describes the testing process for the ATM Banking System, which allows users to perform banking operations like cash withdrawal, balance inquiry, and fund transfer.

◆ Purpose

To ensure the ATM system works accurately, securely, and reliably.

◆ Objectives and Tasks

- Validate all ATM transactions
- Ensure secure authentication (PIN)
- Detect and fix defects

◆ Scope

In Scope:

- Card validation
- PIN authentication
- Cash withdrawal
- Balance inquiry
- Fund transfer

Out of Scope:

- Bank backend processing
- Third-party network systems

◆ Test Strategy

- Functional Testing
- Security Testing
- Performance Testing
- Usability Testing

◆ Test Items

- ATM machine interface
- Card reader
- Transaction processing module

◆ Software Risk Issues

- Incorrect cash dispensing
- Unauthorized access
- Network failure

◆ Features to be Tested

- Valid/Invalid PIN
- Cash withdrawal limits
- Balance check accuracy
- Transaction receipt

◆ Features Not to be Tested

- Internal banking system logic
- External payment networks

◆ Approach

- Design test cases
- Execute transactions
- Record results
- Fix defects and retest

◆ Item Pass / Fail Criteria

- Pass: Correct transaction and output
- Fail: Any mismatch or error

⬙ Suspension Criteria and Resumption

- Suspend if ATM crashes or network fails
- Resume after issue resolution

⬙ Test Deliverables

- Test Plan
- Test Cases
- Defect Reports
- Test Summary Report

⬙ Remaining Test Tasks

- Regression testing
- Final system validation

⬙ Environmental Needs

- ATM simulator / machine
- Bank server connection
- Test cards and PINs

⬙ Staffing and Training Needs

- QA testers
- Training on ATM operations

⬙ Responsibilities

- Tester: Execute test cases
- Developer: Fix issues
- Test Lead: Supervise testing

⬙ Schedule

- Planning: 2 days
- Test Case Design: 3 days
- Execution: 5 days
- Retesting: 2 days

⬙ Planning Risks and Contingencies

- Risk: Network failure → Backup connection
- Risk: Hardware failure → Use simulator

⬙ Approvals

- Test Manager
- Project Manager
- Client

Q2) Design Manual test cases in excel for **Online Payment Gateway** (success, failure, timeout cases). 10

TC ID	Test Case	Steps	Test Data	Expected Result	Actual Result	Status
TC_01	Successful payment	Enter valid card details → Pay	Valid card	Payment success message	—	—
TC_02	Invalid card number	Enter wrong card	1234	Error message	—	—
TC_03	Expired card	Enter expired date	01/20	Payment failed	—	—
TC_04	Wrong CVV	Enter incorrect CVV	000	Payment failed	—	—
TC_05	Insufficient balance	Use low balance card	—	Payment failed	—	—
TC_06	Payment timeout	Delay response	—	Timeout message	—	—
TC_07	Network failure	Disconnect internet	—	Error/Retry option	—	—
TC_08	Double click pay	Click twice	—	Only one transaction processed	—	—
TC_09	Cancel payment	Click cancel	—	Transaction cancelled	—	—

TC ID	Test Case	Steps	Test Data	Expected Result	Actual Result	Status
TC_10	OTP validation	Enter correct OTP	123456	Payment success	—	—

Slip 11

Q1) Design a **Test Plan for Library Management System**. 15

◆ Test Plan Identifier

TP_LMS_01

◆ Introduction

This document describes the testing process for the Library Management System, which manages book records, member details, issue/return of books, and fines.

◆ Purpose

To ensure the system works correctly, efficiently, and meets user requirements.

◆ Objectives and Tasks

- Verify all modules (books, members, transactions)
- Ensure accurate issue/return operations
- Detect and fix defects

◆ Scope

In Scope:

- Book management (add, update, delete)
- Member registration
- Issue/Return books

- Fine calculation

Out of Scope:

- External payment systems
- Third-party integrations

⬙ Test Strategy

- Functional Testing
- Integration Testing
- System Testing
- Performance Testing

⬙ Test Items

- Book module
- Member module
- Transaction module

⬙ Software Risk Issues

- Data loss
- Incorrect fine calculation
- System crash

⬙ Features to be Tested

- Add/Search books
- Member registration
- Issue/Return process
- Fine calculation accuracy

⬙ Features Not to be Tested

- External APIs
- Payment gateway logic

◆ Approach

- Design test cases
- Execute test cases
- Record defects
- Perform retesting

◆ Item Pass / Fail Criteria

- Pass: Output matches expected result
- Fail: Any mismatch

◆ Suspension Criteria and Resumption

- Suspend if system crashes
- Resume after bug fix

◆ Test Deliverables

- Test Plan
- Test Cases
- Defect Reports
- Test Summary Report

◆ Remaining Test Tasks

- Regression testing
- Final validation

◆ Environmental Needs

- Computer system
- Database server
- Application software

◆ Staffing and Training Needs

- QA testers
- Training on system modules

◆ Responsibilities

- Tester: Execute test cases
- Developer: Fix bugs
- Test Lead: Monitor testing

◆ Schedule

- Planning: 2 days
- Test Case Design: 3 days
- Execution: 5 days
- Retesting: 2 days

◆ Planning Risks and Contingencies

- Risk: Data loss → Backup system
- Risk: Delay → Adjust schedule

◆ Approvals

- Test Manager
- Project Manager
-

Q2) Design Manual test cases in excel for **Mobile App Notifications (enable/disable, alerts)**. 10

TC ID	Test Case	Steps	Test Data	Expected Result	Actual Result	Status
TC_01	Enable notifications	Turn ON notification setting	—	Notifications enabled	—	—

TC ID	Test Case	Steps	Test Data	Expected Result	Actual Result	Status
TC_02	Disable notifications	Turn OFF setting	—	Notifications disabled	—	—
TC_03	Receive notification	Trigger event	—	Notification received	—	—
TC_04	Notification alert sound	Receive notification	—	Sound plays	—	—
TC_05	Notification vibration	Receive notification	—	Vibration occurs	—	—
TC_06	Notification click	Tap notification	—	Opens correct screen	—	—
TC_07	Background notification	App in background	—	Notification received	—	—
TC_08	Foreground notification	App open	—	Alert displayed	—	—
TC_09	Multiple notifications	Trigger many alerts	—	All notifications shown	—	—
TC_10	Clear notification	Swipe notification	—	Notification removed	—	—

Slip 10

Q1) Write a **Test Plan for Mobile Calculator Application**. 15M

- Test Plan Identifier

TP_CALC_01

- Introduction

This document describes the testing process for the Mobile Calculator Application, used to perform arithmetic operations like addition, subtraction, multiplication, and division.

⬙ Purpose

To ensure the calculator performs accurate calculations and works efficiently on mobile devices.

⬙ Objectives and Tasks

- Verify all arithmetic operations
- Ensure correct input/output behavior
- Detect and fix defects

⬙ Scope

In Scope:

- Basic operations (+, −, ×, ÷)
- Input validation
- UI buttons functionality

Out of Scope:

- Scientific calculations (if not included)
- External integrations

⬙ Test Strategy

- Functional Testing
- UI Testing
- Usability Testing
- Performance Testing

⬙ Test Items

- Calculator UI
- Input keypad
- Calculation logic

⬙ Software Risk Issues

- Incorrect calculations
- App crash
- Input errors

⬙ Features to be Tested

- Addition, subtraction, multiplication, division
- Decimal input
- Clear/reset function

⬙ Features Not to be Tested

- Advanced/scientific features (if not present)

⬙ Approach

- Design test cases
- Execute operations
- Compare results
- Fix and retest

⬙ Item Pass / Fail Criteria

- Pass: Correct calculation result
- Fail: Incorrect result or crash

⬙ Suspension Criteria and Resumption

- Suspend if app crashes
- Resume after fix

⬙ Test Deliverables

- Test Plan

- Test Cases
- Defect Reports
- Test Summary Report

⬙ Remaining Test Tasks

- Regression testing
- Final validation

⬙ Environmental Needs

- Android/iOS mobile device
- Calculator app installed

⬙ Staffing and Training Needs

- QA tester
- Basic training on app usage

⬙ Responsibilities

- Tester: Execute tests
- Developer: Fix bugs
- Test Lead: Supervise

⬙ Schedule

- Planning: 1 day
- Test Case Design: 2 days
- Execution: 3 days
- Retesting: 1–2 days

⬙ Planning Risks and Contingencies

- Risk: App crash → Debug and fix

- Risk: Wrong results → Recheck logic

⋈ Approvals

- Test Manager
- Project Manager

Q2) Write test cases for **Exam Form Submission with mandatory field's validation**. 10M

TC ID	Test Case	Steps	Test Data	Expected Result	Actual Result	Status
TC_01	Submit with all valid data	Fill all fields → Submit	Valid data	Form submitted successfully	—	—
TC_02	Empty form submission	Click submit without input	—	Validation messages shown	—	—
TC_03	Missing name	Leave name field empty	—	Error message	—	—
TC_04	Missing email	Leave email empty	—	Error message	—	—
TC_05	Invalid email	Enter wrong format	test@	Validation error	—	—
TC_06	Missing mobile number	Leave mobile empty	—	Error message	—	—
TC_07	Invalid mobile number	Enter less digits	12345	Validation error	—	—
TC_08	Mandatory checkbox	Do not select checkbox	—	Warning message	—	—
TC_09	Partial form fill	Fill some fields only	—	Validation errors	—	—
TC_10	Field length validation	Enter long input	100 chars	Input restricted/error	—	—

Slip 12

Q1) Write a **Test Plan for Student Management System**. 15

Test Plan Identifier

TP_SMS_01

◈ Introduction

This document describes the testing process for the Student Management System, which manages student records, courses, attendance, and results.

◈ Purpose

To ensure the system functions correctly and maintains accurate student data.

◈ Objectives and Tasks

- Verify all modules (student, course, result)
- Ensure data accuracy and integrity
- Identify and fix defects

◈ Scope

In Scope:

- Student registration
- Course management
- Attendance tracking
- Result processing

Out of Scope:

- External systems
- Payment modules

◆ Test Strategy

- Functional Testing
- Integration Testing
- System Testing
- Performance Testing

◆ Test Items

- Student module
- Course module
- Result module

◆ Software Risk Issues

- Data loss
- Incorrect result calculation
- System crash

◆ Features to be Tested

- Add/update/delete student
- Course enrollment
- Attendance records
- Result generation

◆ Features Not to be Tested

- External APIs
- Third-party integrations

◆ Approach

- Design test cases
- Execute tests
- Log defects
- Retest after fixes

◆ Item Pass / Fail Criteria

- Pass: Expected result matches actual
- Fail: Any mismatch

◆ Suspension Criteria and Resumption

- Suspend if system crashes
- Resume after fix

◆ Test Deliverables

- Test Plan
- Test Cases
- Defect Reports
- Test Summary Report

◆ Remaining Test Tasks

- Regression testing
- Final validation

◆ Environmental Needs

- Computer system
- Database server
- Application software

◆ Staffing and Training Needs

- QA testers
- Training on modules

◆ Responsibilities

- Tester: Execute tests
- Developer: Fix bugs
- Test Lead: Monitor

◆ Schedule

- Planning: 2 days
- Test Case Design: 3 days
- Execution: 5 days
- Retesting: 2 days

◆ Planning Risks and Contingencies

- Risk: Data loss → Backup system
- Risk: Delay → Adjust schedule

◆ Approvals

- Test Manager
- Project Manager
- Client

Q2) Write a Java Program to **check whether a number is prime**. Design test cases for it 10

```
import java.util.Scanner;


public class PrimeCheck {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter number: ");
int n = sc.nextInt();


if (n <= 1) {
    System.out.println("Not Prime");
} else {
    int flag = 1;


    for (int i = 2; i <= n / 2; i++) {
        if (n % i == 0) {
            flag = 0;
            break;
        }
    }


    if (flag == 1)
        System.out.println("Prime Number");
    else
        System.out.println("Not Prime");
}
```

```
    }

}
```

TC ID	Input (n)	Expected Output	Actual Output	Status
TC_01	2	Prime Number	Prime	Pass
TC_02	3	Prime Number	Prime	Pass
TC_03	4	Not Prime	Not Prime	Pass
TC_04	5	Prime Number	Prime	Pass
TC_05	1	Not Prime	Not Prime	Pass
TC_06	0	Not Prime	Not Prime	Pass
TC_07	-7	Not Prime	Not Prime	Pass
TC_08	9	Not Prime	Not Prime	Pass

Slip 13

Q1) Write a Java Program to **check whether a number is prime**. Design test cases for it 10

```
import java.util.Scanner;


public class PrimeCheck {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);



        System.out.print("Enter number: ");

        int n = sc.nextInt();



        if (n <= 1) {
```

```
        System.out.println("Not Prime");
    } else {
        int flag = 1;

        for (int i = 2; i <= n / 2; i++) {
            if (n % i == 0) {
                flag = 0;
                break;
            }
        }

        if (flag == 1)
            System.out.println("Prime Number");
        else
            System.out.println("Not Prime");
    }
  }
}
```

TC ID	Input (n)	Expected Output	Actual Output	Status
TC_01	2	Prime Number	Prime	Pass
TC_02	3	Prime Number	Prime	Pass
TC_03	4	Not Prime	Not Prime	Pass
TC_04	5	Prime Number	Prime	Pass

TC ID	Input (n)	Expected Output	Actual Output	Status
TC_05	1	Not Prime	Not Prime	Pass
TC_06	0	Not Prime	Not Prime	Pass
TC_07	-7	Not Prime	Not Prime	Pass
TC_08	9	Not Prime	Not Prime	Pass

Q2) Write a **Test Plan for Mobile Banking Application**. 15

◆ Test Plan Identifier

TP_MBANK_01

◆ Introduction

This document describes the testing process for the Mobile Banking Application, which allows users to perform banking activities such as login, balance inquiry, fund transfer, bill payment, and transaction history.

◆ Purpose

To ensure the application is secure, reliable, and functions correctly for all banking operations.

◆ Objectives and Tasks

- Validate all banking functionalities
- Ensure secure authentication (OTP, PIN)
- Identify and fix defects

◆ Scope

In Scope:

- User login/logout
- Balance inquiry

- Fund transfer
- Bill payments
- Transaction history

Out of Scope:

- Internal bank server logic
- Third-party payment systems

⬙ Test Strategy

- Functional Testing
- Security Testing
- Performance Testing
- Usability Testing

⬙ Test Items

- Login module
- Transaction module
- Payment module
- Notification module

⬙ Software Risk Issues

- Unauthorized access
- Transaction failure
- Data leakage
- Network issues

⬙ Features to be Tested

- Login with valid/invalid credentials
- OTP verification
- Fund transfer accuracy
- Payment success/failure
- Session timeout

◈ Features Not to be Tested

- Backend banking systems
- External APIs internal logic

◈ Approach

- Design test cases
- Execute transactions
- Record results
- Perform regression testing

◈ Item Pass / Fail Criteria

- Pass: Transaction completed successfully with correct result
- Fail: Any incorrect or failed transaction

◈ Suspension Criteria and Resumption

- Suspend testing if app crashes or critical defect occurs
- Resume after issue is fixed

◈ Test Deliverables

- Test Plan
- Test Cases
- Defect Reports
- Test Summary Report

◈ Remaining Test Tasks

- Regression testing
- Final validation

◆ Environmental Needs

- Android/iOS mobile device
- Internet connection
- Test bank account

◆ Staffing and Training Needs

- QA testers
- Training on banking workflows

◆ Responsibilities

- Tester: Execute test cases
- Developer: Fix defects
- Test Lead: Monitor testing

◆ Schedule

- Planning: 2 days
- Test Case Design: 3 days
- Execution: 5 days
- Retesting: 2 days

◆ Planning Risks and Contingencies

- Risk: Network failure → Use stable connection
- Risk: Security issue → Immediate fix and retest

◆ Approvals

- Test Manager
- Project Manager
- Client

Slip 14

Q1) Design a **Test Plan for Online Examination System**. 15

◆ Test Plan Identifier

TP_OES_01

◆ Introduction

This document describes the testing process for the Online Examination System, which allows students to register, login, take exams, and view results online.

◆ Purpose

To ensure the system works correctly, securely, and handles exams without errors.

◆ Objectives and Tasks

- Validate exam functionality
- Ensure secure login and access
- Detect and fix defects

◆ Scope

In Scope:

- Student registration & login
- Exam start/submit
- Timer functionality
- Result generation

Out of Scope:

- External integrations
- Third-party services

◆ Test Strategy

- Functional Testing
- Security Testing
- Performance Testing
- Usability Testing

◆ Test Items

- Login module
- Exam module
- Result module

◆ Software Risk Issues

- System crash during exam
- Data loss
- Unauthorized access

◆ Features to be Tested

- Login with valid/invalid credentials
- Exam timer accuracy
- Auto-submit on timeout
- Result calculation

◆ Features Not to be Tested

- External systems
- Payment modules

◆ Approach

- Design test cases
- Execute test cases
- Log defects

- Retest after fixes

⬙ Item Pass / Fail Criteria

- Pass: System works as expected
- Fail: Any incorrect behavior

⬙ Suspension Criteria and Resumption

- Suspend if system crashes
- Resume after fix

⬙ Test Deliverables

- Test Plan
- Test Cases
- Defect Reports
- Test Summary Report

⬙ Remaining Test Tasks

- Regression testing
- Final validation

⬙ Environmental Needs

- Computer/mobile device
- Internet connection
- Test server

⬙ Staffing and Training Needs

- QA testers
- Training on exam system

⬙ Responsibilities

- Tester: Execute tests
- Developer: Fix bugs
- Test Lead: Monitor

⬙ Schedule

- Planning: 2 days
- Test Case Design: 3 days
- Execution: 5 days
- Retesting: 2 days

⬙ Planning Risks and Contingencies

- Risk: Server crash → Backup server
- Risk: Network failure → Retry mechanism

⬙ Approvals

- Test Manager
- Project Manager

Q2) Prepare a defect report for **Login fails with valid credentials**. 10

- **Bug ID:** DEF_001
- **Title:** Login fails with valid credentials
- **Description:** User is unable to login even after entering correct username and password
- **Steps to Reproduce:**
  - Open login page
  - Enter valid username and password
  - Click Login button
- **Expected Result:** User should login successfully
- **Actual Result:** Login fails and error message is displayed

- **Severity:** High
- **Priority:** High
- **Environment:** Chrome browser, Windows OS
- **Status:** Open
- **Assigned:** Developer

Slip 15

Q1)Design entry and exit criteria for test case, design Manual test cases in excel for E- Commerce shopping portal's Login form (like Flipkart, Amazon) 10

TC ID	Test Case	Steps	Test Data	Expected Result	Actual Result	Status
TC_01	Valid login	Enter username & password → Login	user123 / pass123	Login successful	—	—
TC_02	Invalid password	Valid username + wrong password	user123 / wrong	Error message	—	—
TC_03	Invalid username	Wrong username + valid password	wrong / pass123	Error message	—	—
TC_04	Empty fields	Click login without input	—	Validation message	—	—
TC_05	Only username	Enter username only	user123	Password required	—	—
TC_06	Only password	Enter password only	pass123	Username required	—	—
TC_07	Password masking	Enter password	****	Password hidden	—	—
TC_08	Case sensitivity	Enter USER123	—	System validates correctly	—	—
TC_09	Forgot password	Click link	—	Reset page opens	—	—
TC_10	SQL injection	Enter ' OR 1=1	—	Input rejected	—	—

Q2) Design a **Test Plan for Hospital Management System**. 15

◆ Test Plan Identifier

TP_HMS_01

◆ Introduction

This document describes the testing process for the Hospital Management System, which manages patient records, doctor details, appointments, billing, and reports.

◆ Purpose

To ensure the system works accurately, securely, and efficiently in managing hospital operations.

◆ Objectives and Tasks

- Verify all modules (patient, doctor, billing)
- Ensure data accuracy and security
- Detect and fix defects

◆ Scope

In Scope:

- Patient registration
- Doctor management
- Appointment scheduling
- Billing system

Out of Scope:

- External insurance systems
- Third-party integrations

◆ Test Strategy

- Functional Testing

- Integration Testing
- System Testing
- Security Testing

◈ Test Items

- Patient module
- Appointment module
- Billing module

◈ Software Risk Issues

- Data loss
- Incorrect billing
- Unauthorized access

◈ Features to be Tested

- Patient registration
- Appointment booking
- Billing accuracy
- Record management

◈ Features Not to be Tested

- External APIs
- Third-party systems

◈ Approach

- Design test cases
- Execute tests
- Log defects
- Retest after fixes

⬙ Item Pass / Fail Criteria

- Pass: Output matches expected
- Fail: Any mismatch

⬙ Suspension Criteria and Resumption

- Suspend if system crashes
- Resume after fix

⬙ Test Deliverables

- Test Plan
- Test Cases
- Defect Reports
- Test Summary Report

⬙ Remaining Test Tasks

- Regression testing
- Final validation

⬙ Environmental Needs

- Computer system
- Database server
- Application software

⬙ Staffing and Training Needs

- QA testers
- Training on modules

⬙ Responsibilities

- Tester: Execute tests

- Developer: Fix bugs
- Test Lead: Monitor

◆ Schedule

- Planning: 2 days
- Test Case Design: 3 days
- Execution: 5 days
- Retesting: 2 days

◆ Planning Risks and Contingencies

- Risk: Data loss → Backup
- Risk: Delay → Adjust schedule

◆ Approvals

- Test Manager
- Project Manager

Slip 16

Q1) Design a **Test Plan for Chat / Messaging Application**. 15

◆ Test Plan Identifier

TP_CHAT_01

◆ Introduction

This document describes the testing process for the Chat/Messaging Application which allows users to send and receive messages, media files, and notifications in real time.

◆ Purpose

To ensure the application works correctly, securely, and delivers messages without delay or loss.

◆ Objectives and Tasks

- Verify messaging functionality
- Ensure real-time message delivery
- Identify and fix defects

◆ Scope

In Scope:

- User login/logout
- One-to-one chat
- Group chat
- Media sharing
- Notifications

Out of Scope:

- Third-party integrations
- Backend server logic

◆ Test Strategy

- Functional Testing
- Performance Testing
- Security Testing
- Usability Testing

◆ Test Items

- Login module
- Chat module
- Notification module
- Media sharing module

◆ Software Risk Issues

- Message delay or loss
- App crash
- Unauthorized access
- Network dependency

◆ Features to be Tested

- Sending/receiving messages
- Typing indicators
- Message delivery status (sent/delivered/read)
- Notifications
- Media file upload/download

◆ Features Not to be Tested

- Internal server processing
- External APIs

◆ Approach

- Design test cases
- Execute tests
- Record defects
- Perform regression testing

◆ Item Pass / Fail Criteria

- Pass: Message sent/received correctly
- Fail: Message delay, failure, or wrong behavior

◆ Suspension Criteria and Resumption

- Suspend if app crashes or server fails
- Resume after issue is resolved

◈ Test Deliverables

- Test Plan
- Test Cases
- Defect Reports
- Test Summary Report

◈ Remaining Test Tasks

- Regression testing
- Final validation

◈ Environmental Needs

- Mobile device (Android/iOS)
- Internet connection
- Test accounts

◈ Staffing and Training Needs

- QA testers
- Training on chat features

◈ Responsibilities

- Tester: Execute test cases
- Developer: Fix defects
- Test Lead: Monitor testing

◈ Schedule

- Planning: 2 days
- Test Case Design: 3 days
- Execution: 5 days
- Retesting: 2 days

❖ Planning Risks and Contingencies

- Risk: Network failure → Use stable internet
- Risk: Server downtime → Backup server

❖ Approvals

- Test Manager
- Project Manager

Q2) Write a defect report for **Calculator showing wrong addition result**. 10

1. **Bug ID:** DEF_002
2. **Title:** Calculator shows incorrect addition result
3. **Description:** Calculator returns wrong result when adding two numbers
4. **Steps to Reproduce:**
    - Open calculator
    - Enter first number (e.g., 10)
    - Enter second number (e.g., 5)
    - Click '+' and '='
5. **Expected Result:** Output should be 15
6. **Actual Result:** Output shows incorrect value (e.g., 14)
7. **Severity:** High
8. **Priority:** High
9. **Environment:** Windows OS, Calculator Application
10. **Status:** Open
11. **Assigned:** Developer

Q1) Write a **Test Plan for Inventory Management System**. 10

1. Test Plan Identifier
IMS_TP_01

2. Introduction
This document describes the testing approach for the Inventory Management System used to manage products, stock, suppliers, and orders.

3. Purpose
To ensure the system works correctly and meets business requirements.

4. Objectives and Tasks

- Verify all modules
- Detect defects
- Ensure system reliability

5. Scope
Testing includes product management, stock updates, supplier details, and order processing.

6. Test Strategy

- Manual Testing
- Functional Testing
- Integration Testing
- System Testing

7. Test Items

- Login Module
- Product Module
- Stock Module
- Supplier Module
- Reports

8. Software Risk Issues

- Data loss
- Incorrect stock update
- System crash

9. Features to be Tested

- Add/Edit/Delete products
- Stock management
- Order processing
- Report generation

10. Features Not to be Tested

- External systems
- Hardware components

11. Approach

- Design test cases
- Execute manually
- Record results
- Report bugs

12. Item Pass / Fail Criteria

- Pass: Correct output
- Fail: Incorrect output

13. Suspension Criteria and Resumption

- Suspend if major defect found
- Resume after fix

14. Test Deliverables

- Test Plan
- Test Cases
- Defect Reports
- Test Summary

15. Remaining Test Tasks

- Regression testing
- Performance testing

16. Environmental Needs (Test Environment)

- OS: Windows/Linux
- DB: MySQL/PostgreSQL
- Browser: Chrome

17. Staffing and Training Needs

- Skilled testers
- Basic system training

18. Responsibilities

- Tester: Testing
- Developer: Bug fixing
- Manager: Monitoring

19. Schedule

- Planning: 2 days
- Execution: 5 days
- Reporting: 2 days

20. Planning Risks and Contingencies

- Risk: Delay
- Solution: Adjust timeline

Q2) Write a java Program to **reverse 3 digit number**. Write Test cases for this code. 15

```
import java.util.Scanner;


public class Reverse3Digit {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);


System.out.print("Enter a 3-digit number: ");

int num = sc.nextInt();
```

```
if (num < 100 || num > 999) {
System.out.println("Invalid input! Enter a 3-digit number.");
} else {
int reverse = 0;

while (num != 0) {
int digit = num % 10;
reverse = reverse * 10 + digit;
num = num / 10;
}

System.out.println("Reversed number: " + reverse);
}
}
}
```

Test Case ID	Input	Expected Output	Actual Output	Status
TC01	123	321	321	Pass
TC02	456	654	654	Pass
TC03	100	1	1	Pass
TC04	999	999	999	Pass
TC05	120	21	21	Pass
TC06	12	Invalid input	Invalid	Pass
TC07	1000	Invalid input	Invalid	Pass

Test Case ID	Input	Expected Output	Actual Output	Status
TC08	-123	Invalid input	Invalid	Pass

Slip 18

Q1) Design test cases for a program to **accept only numeric input**. 10

Test Case ID	Input	Description	Expected Output	Actual Output	Status
TC01	123	Valid integer	Accepted	Accepted	Pass
TC02	0	Zero value	Accepted	Accepted	Pass
TC03	-45	Negative number	Accepted (if allowed)	Accepted	Pass
TC04	12.5	Decimal number	Accepted (if allowed)	Accepted	Pass
TC05	abc	Alphabets	Rejected with error	Rejected	Pass
TC06	123abc	Alphanumeric	Rejected	Rejected	Pass
TC07	@#$	Special characters	Rejected	Rejected	Pass
TC08	(blank)	Empty input	Rejected	Rejected	Pass
TC09	" "	Only spaces	Rejected	Rejected	Pass
TC10	999999999	Large numeric value	Accepted	Accepted	Pass

Q2) Write a Java Program to calculate maximum of given 2 numbers. Design test cases for this code. 15

```
import java.util.Scanner;


public class MaxOfTwo {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);


      System.out.print("Enter first number: ");
```

```
        int a = sc.nextInt();

        System.out.print("Enter second number: ");
        int b = sc.nextInt();

        if (a > b) {
            System.out.println("Maximum number is: " + a);
        } else if (b > a) {
            System.out.println("Maximum number is: " + b);
        } else {
            System.out.println("Both numbers are equal");
        }
    }
}
```

Test Case ID	Input (a, b)	Expected Output	Actual Output	Status
TC01	(10, 20)	20	20	Pass
TC02	(30, 15)	30	30	Pass
TC03	(0, 0)	Both equal	Both equal	Pass
TC04	(-5, -2)	-2	-2	Pass
TC05	(-3, 7)	7	7	Pass
TC06	(100, 100)	Both equal	Both equal	Pass
TC07	(Integer.MAX, Integer.MIN)	MAX value	MAX value	Pass

Q1) Design a **Test Plan for Online Payment System**. 15

1. Test Plan Identifier
OPS_TP_01

2. Introduction
This document defines the testing strategy for the Online Payment System used for secure online transactions.

3. Purpose
To ensure accurate, secure, and reliable payment processing.

4. Objectives and Tasks

- Verify payment functionality
- Ensure security of transactions
- Identify and fix defects

5. Scope
Includes login, payment processing, transaction history, and notifications.

6. Test Strategy

- Functional Testing
- Security Testing
- Integration Testing
- Performance Testing

7. Test Items

- Login Module
- Payment Gateway
- Transaction Module
- Notification System

8. Software Risk Issues

- Transaction failure
- Data leakage
- Duplicate payments
- Network issues

9. Features to be Tested

- User authentication
- Payment via Debit/Credit/UPI
- Transaction status
- Receipt generation

10. Features Not to be Tested

- Bank internal systems
- Third-party gateway internals

11. Approach

- Prepare test cases
- Execute test scenarios
- Log defects
- Retest after fixes

12. Item Pass / Fail Criteria

- Pass: Payment successful
- Fail: Payment failed or incorrect

13. Suspension Criteria and Resumption

- Suspend testing if critical defect occurs
- Resume after fixing issues

14. Test Deliverables

- Test Plan
- Test Cases
- Defect Reports
- Test Summary Report

15. Remaining Test Tasks

- Load testing
- Security testing

16. Environmental Needs (Test Environment)

- OS: Windows/Linux
- Browser: Chrome/Firefox
- Payment sandbox

17. Staffing and Training Needs

- Skilled testers
- Security awareness training

18. Responsibilities

- Testers: Execute testing
- Developers: Fix bugs
- Manager: Monitor progress

19. Schedule

- Planning: 2 days
- Execution: 6 days
- Reporting: 2 days

20. Planning Risks and Contingencies

- Risk: Server downtime
- Solution: Backup system

Q2) Write a Java Program **swap two numbers**. Design test cases for this code. 10

```
import java.util.Scanner;



public class SwapNumbers {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);



      System.out.print("Enter first number: ");

      int a = sc.nextInt();
```

```
        System.out.print("Enter second number: ");

        int b = sc.nextInt();



        int temp;

        temp = a;

        a = b;

        b = temp;



        System.out.println("After swapping:");

        System.out.println("a = " + a + ", b = " + b);

    }

}
```

Test Case ID	Input (a, b)	Expected Output (a, b)	Actual Output	Status
TC01	(10, 20)	(20, 10)	(20, 10)	Pass
TC02	(5, 5)	(5, 5)	(5, 5)	Pass
TC03	(0, 7)	(7, 0)	(7, 0)	Pass
TC04	(-3, 9)	(9, -3)	(9, -3)	Pass
TC05	(-5, -10)	(-10, -5)	(-10, -5)	Pass
TC06	(1000, 2000)	(2000, 1000)	(2000, 1000)	Pass

Slip 20

Q1) Write a **Test Plan for E-learning Platform**. 15

1. Test Plan Identifier
ELP_TP_01

2. Introduction
This document describes the testing process for the E-learning Platform used for online courses, video lectures, and assessments.

3. Purpose
To ensure the platform works efficiently, securely, and provides a smooth learning experience.

4. Objectives and Tasks

- Verify all modules functionality
- Ensure performance and usability
- Detect and fix defects

5. Scope
Includes user registration, login, course access, video streaming, quizzes, and certificates.

6. Test Strategy

- Functional Testing
- Usability Testing
- Performance Testing
- Security Testing

7. Test Items

- Login/Register Module
- Course Module
- Video Player
- Quiz/Assessment Module
- Certificate Generation

8. Software Risk Issues

- Server downtime
- Video buffering issues
- Data loss
- Unauthorized access

9. Features to be Tested

- User login and registration

- Course enrollment
- Video playback
- Quiz submission
- Result generation

10. Features Not to be Tested

- Third-party video hosting internals
- External payment systems

11. Approach

- Prepare test cases
- Execute test scenarios
- Record results
- Report defects

12. Item Pass / Fail Criteria

- Pass: Expected output achieved
- Fail: System does not meet requirements

13. Suspension Criteria and Resumption

- Suspend if critical errors occur
- Resume after issue resolution

14. Test Deliverables

- Test Plan
- Test Cases
- Defect Reports
- Test Summary Report

15. Remaining Test Tasks

- Regression testing
- Load testing

16. Environmental Needs (Test Environment)

- OS: Windows/Linux
- Browser: Chrome/Firefox
- Internet connection

17. Staffing and Training Needs

- Skilled testers
- Training on platform usage

18. Responsibilities

- Testers: Execute testing
- Developers: Fix bugs
- Manager: Monitor progress

19. Schedule

- Planning: 2 days
- Execution: 6 days
- Reporting: 2 days

20. Planning Risks and Contingencies

- Risk: Server crash
- Solution: Backup server

Q2) Write a defect report for **Application crashes on clicking Submit button**. 10

Defect Report

1. Defect ID:
DEF_001

2. Defect Title:
Application crashes on clicking Submit button

3. Description:
The application crashes when the user clicks the Submit button after entering data in the form.

4. Module:
Form Submission Module

5. Severity:
Critical

6. Priority:
High

7. Environment:

- OS: Windows 10
- Browser: Chrome

8. Steps to Reproduce:

1. Open the application
2. Navigate to form page
3. Enter valid data
4. Click on Submit button

9. Expected Result:
Form should be submitted successfully and confirmation message displayed.

10. Actual Result:
Application crashes and closes unexpectedly.

11. Status:
Open

Slip 21

Q1) Prepare a bug report for **ATM allows withdrawal without sufficient balance**. 10

1. Bug ID:
ATM_001

2. Bug Title:
ATM allows withdrawal without sufficient balance

3. Description:
The ATM system allows users to withdraw money even when the account balance is less than the withdrawal amount.

4. Module:
Withdrawal Module

5. Severity:
Critical

6. Priority:
High

7. Environment:

- ATM Machine / Banking System
- OS: Embedded System / Windows
- Network: Online

8. Steps to Reproduce:

1. Insert ATM card
2. Enter valid PIN
3. Select Withdrawal option
4. Enter amount greater than available balance
5. Click Confirm

9. Expected Result:
System should display “Insufficient Balance” and prevent transaction.

10. Actual Result:
System allows withdrawal and dispenses money.

11. Status:
Open

12. Reported By:
Tester

13. Assigned To:
Developer

Q2) Write a java Program to calculate Grades for students as per marks scored . Write Test cases for this code.

15M

```
import java.util.Scanner;
```

```
public class StudentGrade {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);



        System.out.print("Enter marks: ");

        int marks = sc.nextInt();



        if (marks >= 90 && marks <= 100) {

            System.out.println("Grade: A");

        } else if (marks >= 75) {

            System.out.println("Grade: B");

        } else if (marks >= 60) {

            System.out.println("Grade: C");

        } else if (marks >= 50) {

            System.out.println("Grade: D");

        } else if (marks >= 0) {

            System.out.println("Grade: Fail");

        } else {

            System.out.println("Invalid Marks");

        }
```

```
  }

}
```

Test Case ID	Input Marks	Expected Output	Actual Output	Status
TC01	95	Grade A	A	Pass
TC02	80	Grade B	B	Pass
TC03	65	Grade C	C	Pass
TC04	55	Grade D	D	Pass
TC05	40	Fail	Fail	Pass
TC06	100	Grade A	A	Pass
TC07	0	Fail	Fail	Pass
TC08	-5	Invalid Marks	Invalid	Pass
TC09	110	Invalid Marks (optional validation)	Not handled	Fail

---

Slip 22

Q1) Design a **Test Plan for Desktop Notepad Application**. 15

1. Test Plan Identifier
DN_TP_01

2. Introduction
This document describes the testing process for the Desktop Notepad Application used for creating, editing, and saving text files.

3. Purpose
To ensure the application works correctly and provides a smooth user experience.

4. Objectives and Tasks

- Verify all features of Notepad
- Ensure stability and usability
- Detect and fix defects

5. Scope
Includes file operations, editing features, and basic formatting.

6. Test Strategy

- Functional Testing
- Usability Testing
- System Testing

7. Test Items

- File Menu (New, Open, Save, Save As)
- Edit Menu (Cut, Copy, Paste, Undo, Redo)
- Format Options
- Print Option

8. Software Risk Issues

- Data loss
- File corruption
- Application crash

9. Features to be Tested

- Create new file
- Open existing file
- Save file
- Edit text (cut, copy, paste)
- Print document

10. Features Not to be Tested

- OS-level features
- External printer hardware

11. Approach

- Design test cases
- Execute manually
- Record results
- Report defects

12. Item Pass / Fail Criteria

- Pass: Expected output achieved
- Fail: Incorrect behavior

13. Suspension Criteria and Resumption

- Suspend if major crash occurs
- Resume after fix

14. Test Deliverables

- Test Plan
- Test Cases
- Defect Reports
- Test Summary Report

15. Remaining Test Tasks

- Regression testing
- Performance testing

16. Environmental Needs (Test Environment)

- OS: Windows/Linux
- RAM: Minimum 2GB

17. Staffing and Training Needs

- Basic tester knowledge
- No special training required

18. Responsibilities

- Tester: Execute testing
- Developer: Fix issues
- Manager: Monitor progress

19. Schedule

- Planning: 1–2 days
- Execution: 3–4 days
- Reporting: 1–2 days

20. Planning Risks and Contingencies

- Risk: System crash
- Solution: Backup and recovery

Q2) Write a bug report for **Duplicate records created in database**. 10

1. Bug ID:
DB_001

2. Bug Title:
Duplicate records created in database

3. Description:
The system creates duplicate entries in the database when the user submits the same form multiple times.

4. Module:
Database / Data Entry Module

5. Severity:
High

6. Priority:
High

7. Environment:

- OS: Windows/Linux
- Database: MySQL/PostgreSQL
- Browser: Chrome

8. Steps to Reproduce:

1. Open application
2. Navigate to data entry form
3. Enter valid details
4. Click Submit button multiple times quickly

9. Expected Result:
Only one record should be stored in the database.

10. Actual Result:
Multiple duplicate records are created.

11. Status:
Open

12. Reported By:
Tester

13. Assigned To:
Developer

Slip 23

Q1) Design entry and exit criteria for test case, design Manual test cases in excel for E- Commerce shopping portal's **Registration** form (like Flipkart, Amazon) 15

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC01	Verify registration page loads	Open registration page	URL	Page should open successfully		
TC02	Valid user registration	Enter valid details & submit	Name, Email, Password	Account created successfully		
TC03	Empty fields validation	Click submit without data	Blank	Error messages displayed		
TC04	Invalid email format	Enter wrong email	abc@	Error message shown		
TC05	Password length check	Enter short password	123	Error message displayed		
TC06	Password mismatch	Enter different passwords	Pass1, Pass2	Error shown		
TC07	Mobile number validation	Enter invalid number	12345	Error message		
TC08	Duplicate email check	Register with existing email	Existing email	Error message displayed		
TC09	Terms & conditions unchecked	Submit without checking	-	Warning message		

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC10	Successful submission	Fill all valid fields	Valid data	Registration successful		

Q2) Prepare a defect report for **Logout not working properly**. 10

1. Defect ID:
DEF_002

2. Defect Title:
Logout not working properly

3. Description:
The logout functionality is not working correctly. After clicking logout, the user is not redirected to the login page and remains logged in.

4. Module:
Authentication / Logout Module

5. Severity:
High

6. Priority:
High

7. Environment:

- OS: Windows
- Browser: Chrome

8. Steps to Reproduce:

1. Login to application
2. Click on Logout button
3. Observe behavior

9. Expected Result:
User should be logged out and redirected to login page.

10. Actual Result:
User remains logged in or session is not terminated.

11. Status:
Open

12. Reported By:
Tester

13. Assigned To:
Developer

Slip 24

Q1) Write a **Test Plan for Railway Reservation System**. 15

1. Test Plan Identifier
RRS_TP_01

2. Introduction
This document describes the testing process for the Railway Reservation System used for booking, cancelling, and managing train tickets.

3. Purpose
To ensure accurate ticket booking, availability checking, and secure transactions.

4. Objectives and Tasks

- Verify booking and cancellation functionality
- Ensure data accuracy
- Detect and fix defects

5. Scope
Includes login, train search, ticket booking, payment, and cancellation.

6. Test Strategy

- Functional Testing
- Integration Testing
- System Testing
- Performance Testing

7. Test Items

- Login Module
- Train Search Module
- Booking Module
- Payment Module
- Cancellation Module

8. Software Risk Issues

- Overbooking of seats
- Payment failure
- Data inconsistency
- Server downtime

9. Features to be Tested

- User login
- Train availability search
- Ticket booking
- Payment processing
- Ticket cancellation

10. Features Not to be Tested

- External bank systems
- Third-party APIs

11. Approach

- Design test cases
- Execute test scenarios
- Record results
- Report defects

12. Item Pass / Fail Criteria

- Pass: Correct booking/cancellation
- Fail: Incorrect output or failure

13. Suspension Criteria and Resumption

- Suspend if critical issue occurs
- Resume after fix

14. Test Deliverables

- Test Plan
- Test Cases
- Defect Reports
- Test Summary

15. Remaining Test Tasks

- Regression testing
- Load testing

16. Environmental Needs (Test Environment)

- OS: Windows/Linux
- Browser: Chrome/Firefox
- Internet connection

17. Staffing and Training Needs

- Skilled testers
- Training on system usage

18. Responsibilities

- Tester: Testing
- Developer: Bug fixing
- Manager: Monitoring

19. Schedule

- Planning: 2 days
- Execution: 6 days
- Reporting: 2 days

20. Planning Risks and Contingencies

- Risk: Server overload
- Solution: Backup servers

Q2) Write a defect report for **Password field accepting less than minimum characters**.

1. Defect ID:
DEF_003

2. Defect Title:
Password field accepts less than minimum required characters

3. Description:
The system allows users to enter a password with fewer characters than the required minimum length (e.g., less than 8 characters).

4. Module:
User Registration / Login Module

5. Severity:
High

6. Priority:
High

7. Environment:

- OS: Windows
- Browser: Chrome

8. Steps to Reproduce:

1. Open registration/login page
2. Enter password with less than minimum characters (e.g., 4 characters)
3. Submit the form

9. Expected Result:
System should display an error message and prevent submission until a valid password length is entered.

10. Actual Result:
System accepts the password and allows submission.

11. Status:
Open

12. Reported By:
Tester

13. Assigned To:
Developer